DEEP Learning.ai

Neural Networks

Binary Classification

Notation

(x, y) $x \in \mathbb{R}^{n_x}$, $y \in \{0, 1\}$

m training examples: $\{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \ldots, (x^{(m)}, y^{(m)})\}$

$M = m_{train}$ M_{test} = # test examples

$$X = \begin{bmatrix} | & | & & | \\ x^{(1)} & x^{(2)} & \cdots & x^{(m)} \\ | & | & & | \end{bmatrix} \quad n_x \text{ rows}, \; m \text{ columns}$$

$$Y = \begin{bmatrix} y^{(1)} & y^{(2)} & \cdots & y^{(m)} \end{bmatrix}$$

$Y \in \mathbb{R}^{1 \times m}$

$Y.shape = (1, m)$

$X \in \mathbb{R}^{n_x \times m}$

$X.shape = (n_x, m)$

Logistic Regression

Given X, want $\hat{y} = P(y = 1 | x)$ $x \in \mathbb{R}^{n_x}$

$0 \le \hat{y} \le 1$

Parameters: $w \in \mathbb{R}^{n_x}$, $b \in \mathbb{R}$

Output $\hat{y} = \sigma(w^T x + b)$

z

$\sigma(z) = \frac{1}{1 + e^{-z}}$

if z large $\sigma(z) \approx \frac{1}{1+0} = 1$

if z large -ve number $\sigma(z) = \frac{1}{1+e^{-z}} \approx \frac{1}{1+\text{Bignum}} \approx 0$

$x_0 = 1$ $x \in \mathbb{R}^{n_x + 1}$ $\hat{y} = \sigma(\theta^T x)$

$\theta = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_{n_x} \end{bmatrix}$ $\left.\begin{matrix} \\ \\ \\ \\ \end{matrix}\right\}$ n_x $\left.\begin{matrix} b \\ w \end{matrix}\right\}$ $n_x + 1$

Logistic regression cost function

$\hat{y}^{(i)} = \sigma(w^T x^{(i)} + b)$ $\sigma(z^{(i)}) = \frac{1}{1+e^{-z^{(i)}}}$

- loss (error) function $\mathcal{L}(\hat{y}, y) = \frac{1}{2}(\hat{y} - y)^2$

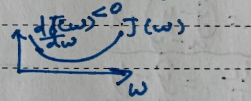
$\mathcal{L}(\hat{y}, y) = -(y \log \hat{y} + (1-y) \log(1-\hat{y}))$

if $y = 1$ $\mathcal{L}(\hat{y}, y) = -\log \hat{y}$ ← want $\log \hat{y}$ large. $\hat{y}$ large.

$y = 0$ $\mathcal{L}(\hat{y}, y) = -\log(1-\hat{y})$ ← want $\log(1-\hat{y})$ large ∴ $\hat{y}$ small

cost function: $J(w, b) = \frac{1}{m} \sum_{i=1}^{m} \mathcal{L}(\hat{y}^{(i)}, y^{(i)}) = -\frac{1}{m} \sum_{i=1}^{m} [y^{(i)} \log \hat{y}^{(i)} + (1-y^{(i)}) \log(1-\hat{y}^{(i)})]$

Gradient descent



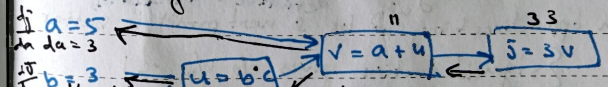
Repeat {

$w := w - \alpha \frac{dJ(w)}{dw}$ $w = w - \alpha \, dw$

$\frac{dJ(w)}{dw} = ?$

backward propagation

Computing derivatives



$\frac{dJ}{du} = s = \frac{dJ}{dv} \cdot \frac{dv}{du}$

$\frac{dJ}{db} = \frac{dJ}{du} \cdot \frac{du}{db} = 3 \cdot b = 1$

$\frac{dJ}{dc} = \frac{dJ}{du} \cdot \frac{du}{dc} = 9$

$u = 6 \to 6.001$

$v = 11 \to 11.001$

$J = 33 \to 33.003$

$b = 3 \to 3.001$

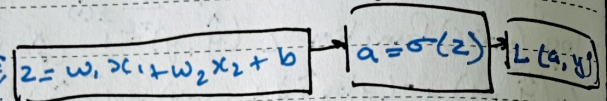
$u = b \cdot c = 6 \to 6.002$

$J = 33.006$

$c = 2$

$v = 11.002$

Logistic regression on m examples derivatives



$dz = \frac{dL}{dz} = \frac{dL(a,y)}{dz}$ $da = \frac{dL(a,y)}{da}$

prediction − actual $= a - y$ $\hat{y} - y$

derivative of $a(1-a)$

$= \frac{dL}{da} \cdot \frac{da}{dz}$ $\longleftrightarrow = \frac{-y}{a} + \frac{1-y}{1-a}$

$\frac{dL}{dw_1} = dw_1 = x_1 \cdot dz$ $dw_2 = x_2 \cdot dz$ $db = dz = 1$

$\frac{dL}{dw} = (\hat{y} - y) \cdot x$ $\frac{dL}{db} = \hat{y} - y$ $a - y$

$w_1 = w_1 - \alpha dw_1$

$w_2 = w_2 - \alpha dw_2$

$b = b - \alpha db$

we trying to find best values of w & b so minimize the loss tells how bad the predictions are

$\frac{dL}{dw}$ it tess if I increase the weight by a little bit, will loss go up or down & by how much using this we do opposite.

$w = w - \alpha \cdot \frac{dL}{dw}$ → slope derivative of increased loss → −ve flips sign loss decrease

Adjust weigh in opp direction of los & next predict is better

{sigmoid function gives o/p b/w of 1

bias b: other factors than x. {Prob may or may not depend on x only}

code structure and flow

(i) initialize_with_zeros(dim): weight & bias to zero

```
w = np.zeros((dim,1))
b = 0
```

(ii) forward_propagation(w, b, X, Y)

```
A = sigmoid(np.dot(w.T, x) + b)
cost = -(1/m) * np.sum(Y * np.log(A) + (1-Y) * np.log(1-A))
```

(iii) Backward propagation : computes gradient

```
dw = (1/m) * np.dot(X, (A-Y).T)
db = (1/m) * np.sum(A-Y)
```

(iv) Optimization : updates parameters using gradient descendent

optimise(w, b, X, Y, num_iterations, learning_rate)

```
w = w - learning_rate * dw
b = b - learning_rate * db
```

(v) Predict(w, b, X): computes predictions on dataset

```
A = sigmoid(np.dot(w.T, x) + b)
Y_prediction = (A >= 0.5)
```

user threshold

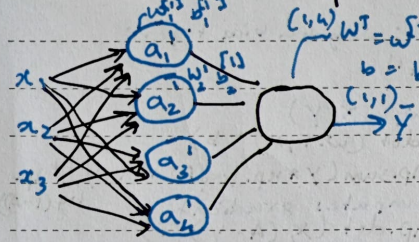
(vi) Model(X_train, Y_train, X_test, Y_test)

```
w, b = initialize_with_zeros(X_train.shape[0])
w, b, grads, cost = optimize(w, b, X_train, Y_tr, learn
                             num_iterat
Y_prediction_train = predict(w, b, X_train)
Y_prediction_test = predict(w, b, X_test)
```

accuracy train: `100 - np.mean(np.abs(Y_predic_train - Y_train)) * 100`

test: `100 - np.mean(np.abs(Y_predic_test - Y_test)) * 100`

Neural Network Representation Learning



$(1,4)$ $W^T = w^{[2]}$ — 2nd layer.

$b = b^{[2]}$

$x \downarrow$

$z^{[1]} = W^{[1]} a^{[0]} + b^{[1]}$

$(4,3)$ $(4,1)$

$a^{[1]} = \sigma(z^{[1]})$

$(4,1)$ $(4,1)$

$z^{[2]} = w^{[2]} a^{[1]} + b^{[2]}$

$a^{[2]} = \sigma(z^{[2]})$

$Z^{[1]} = \begin{bmatrix} | & | & & | \\ z^{[1](1)} & z^{[1](2)} & \cdots & z^{[1](m)} \\ | & | & & | \end{bmatrix}$ ↕ hidden units

↔ training examples.

$X = \begin{bmatrix} | & | & & | \\ x^{(1)} & x^{(2)} & \cdots & x^{(m)} \\ | & | & & | \end{bmatrix}$ ↕ features

(n_x, m)

$W^{[1]} = \begin{bmatrix} — \\ — \\ — \end{bmatrix}$

$z^{[1]} = W^{[1]} X + b^{[1]}$

Activation functions.

(i) sigmoid

(ii) tanh: $a = \frac{e^z - e^{-z}}{e^z + e^{-z}}$

derivative $(1-a^2)$

(iii) ReLu

$a = \max(0, z)$

rectified linear unit

derivative $= \begin{cases} 0, & z \le 0 \\ 1, & z > 0 \end{cases}$

Formulas for computing derivatives

forward propagation

$z^{[1]} = W^{[1]} X + b^{[1]}$

$A^{[1]} = g^{[1]}(z^{[1]})$

$z^{[2]} = W^{[2]} A^{[1]} + b^{[2]}$

$A^{[2]} = g^{[2]}(z^{[2]}) = \sigma(z^{[2]})$

$W^{[2]}$ multiplied with $A^{[1]}$ in final prop

Back propagation

$dz^{[2]} = A^{[2]} - Y$

$Y = [y^{(1)} y^{(2)} \cdots y^{(m)}]$

$(n^{[2]}, m)$

$dW^{[2]} = \frac{1}{m} dz^{[2]} A^{[1]T}$ ← $A^{[1]}$ becomes

$db = \frac{1}{m}$ np.sum $(dz^{[2]}, axis=1, keepdims=True)$ $(n^{[2]}, 1)$

back propagated error — sensitivity of activation

$dz^{[1]} = W^{[2]T} dz^{[2]} * g^{[1]\prime}(z^{[1]})$ $(n^{[1]}, m)$

$(n^{[1]}, m)$ ↑ element wise product

$dW^{[1]} = \frac{1}{m} dz^{[1]} X^T$

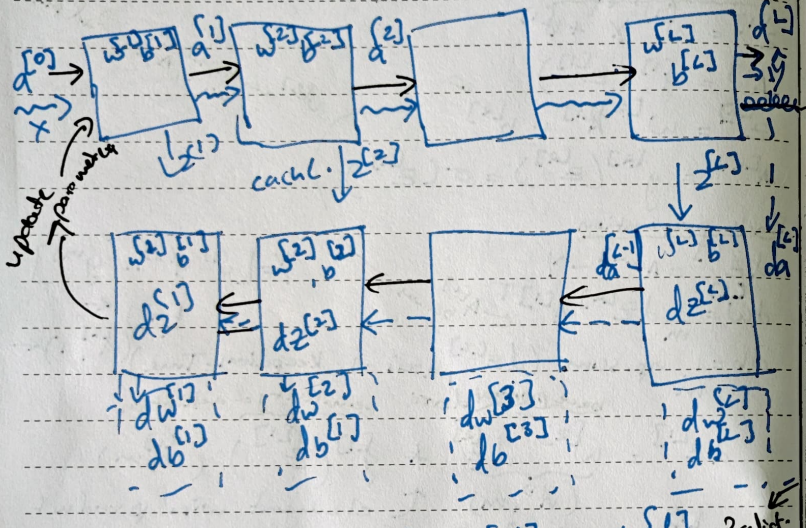
$db^{[1]} = \frac{1}{m}$ np.sum $(dz^{[1]}, axis=1, keepdims=True)$

$(n^{[1]}, 1)$ $(n^{[1]},)$ $(n^{[1]}, n^{[0]})$

$dz^{[1]}$: tells you the error signal at each hidden neuron which is needed to compute how its weights and biases should be updated next.

HQ

forward & Backward function - building blocks



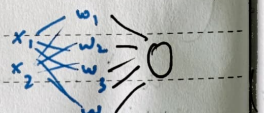
$$W^{[L]} = W^{[L]} - \alpha\, dW^{[L]}$$
$$b^{[L]} = b^{[L]} - \alpha\, db^{[L]}$$

code

n_x = 2 — 2 input features

n_h = 4 → 4 hidden units.

n_y = 1 → 1 output



b1 = [[0.], [0.], [0.], [0.]]

$W_1 = \begin{bmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \\ w_{31} & w_{32} \\ w_{41} & w_{42} \end{bmatrix}$ shape (4, 2) = (n_h, n_x)

$W_2 = [w_1, w_2, w_3, w_4]$ shape (n_y, n_h)

initialize parameters: weights & biases W1 W2 b1 b2

```
W1 = random.randn(n_h, n_x) * 0.01
b1 = np.zeros((n_h, 1))
W2 = np.random.randn(n_y, n_h) * 0.01
b2 = (n_y, 1)
parameters = {"W1": W1, "b1": b1, "W2": W2 ...}
```

Forward Propogation

```
Z1 = np.dot(W1, X) + b1
A1 = np.tanh(Z1)
Z2 = np.dot(W2, A1) + b2
A2 = sigmoid(Z2)
cache = (Z1, A1, Z2, A2)
```

$A_1 = \tanh(Z_1)$ so derivative w.r.t Z_1: $1 - A_1^2$.

$\frac{dL}{dZ_2} = \frac{dL}{dA} \cdot \frac{dA}{dZ_2}$.

Backward propogation

```
dZ2 = A2 - Y.   gradient for o/p layer (sigmoid)
dW2 = (1/m) * np.dot(dZ2, A1.T)   because gradient w.r.t A1 in Z1
db2 = (1/m) * np.sum(dZ2, axis=1, keepdims=True)
dA1 = np.dot(W2.T, dZ2)        dZ2/dA1 = W2 = dL/dZ2 . dZ2/dA1
dZ1 = dA1 * (1 - np.power(A1, 2))
dW1 = (1/m) * np.dot(dZ1, X.T)
db1 = (1/m) * np.sum(dZ1, axis=1)
```

Update parameters (gradient descendent)

```
W1 = W1 - learning_rate * dW1
...
parameters = {"W1": W1 ...}
```

Predictions:

compute Fwd prop.

$Z_1 \to A_1 \to Z_2 \to A_2$.

```
predictions = (A2 > 0.5).astype(int)   → 1 else 0
```

model:

```
for i in range(0, num_iterations):
    A2, cache = forward_prop(X_train, parameters)
    cost = compute_cost(A2, Y_train)
    grads = backward_prop(X_train, Y_train, cache, parameters)
    parameters = update(parameters, grads, learning_rate)
    if i % 100 == 0: print cost
```

compute cost.

$-\frac{1}{m} * np.sum(Y * np.log(A2) + (1-Y) * np.log(1-A2))$

DEEPLEARNING.AI - Coursera

Course 2 : Improving Deep Neural Networks:
Hyperparameter tuning, Regularization & Optimization

train	dev	test
data: 60%	20%	20%

Basic recipe for ML

- High bias (training data performance)
  - try: Bigger Network, train longer, (NN architecture search)
- High variance (dev set performance)
  - Done
  - try: More data, Regularization, (NN architecture search)

"Bias Variance trade off"

Regularization

$\min_{w,b} J(w,b) \quad w \in \mathbb{R}^{n_x}, b \in \mathbb{R}$ — λ: Regularization parameter

$$J(w,b) = \frac{1}{m}\sum_{i=1}^{m} L(\hat{y}^{(i)}, y^{(i)}) + \frac{\lambda}{2m}\|w\|_2^2$$

L_2 regularization $\|w\|_2^2 = \sum_{j=1}^{n_x} w_j^2 = w^T w$

$$J(w^{[1]}, b^{[1]}, \dots, w^{[L]}, b^{[L]}) = \frac{1}{m}\sum_{i=1}^{m} L(\hat{y}^{(i)}, y^{(i)}) + \frac{\lambda}{2m}\sum_{l=1}^{L} \|w^{[l]}\|_F^2$$

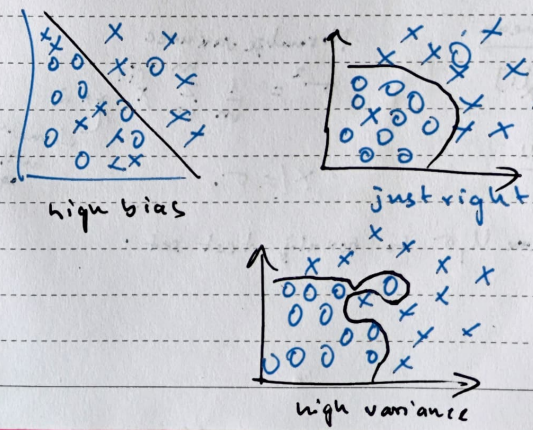
$$\|w^{[l]}\|^2 = \sum_{i=1}^{n^{[l]}} \sum_{j=1}^{n^{[l-1]}} (w_{ij}^{[l]})^2 \qquad w: (n^{[l]}, n^{[l-1]})$$

"Frobenius norm" $\Rightarrow \|\cdot\|_F$

$$dw^{[l]} = [(\text{From backprop}) + \frac{\lambda}{m} w^{[l]}]$$
$$w^{[l]} = w^{[l]} - \alpha\, dw^{[l]}$$

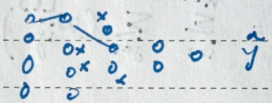
"weight decay"

$$w^{[l]} = w^{[l]} - \alpha[(\text{from backprop}) + \frac{\lambda}{m} w^{[l]}]$$
$$\left(1 - \frac{\alpha\lambda}{m}\right) w^{[l]} - \alpha(\text{from backprop})$$

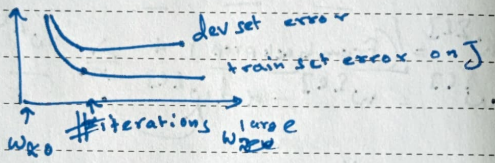


Dropout: regularization method.

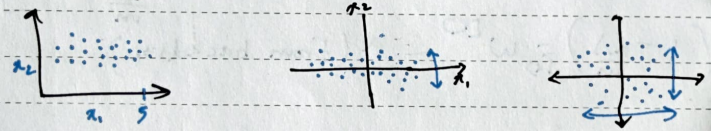
J is not well defined



Early stopping:



Normalizing



Subtract mean

$\mu = \frac{1}{m}\sum_{i=1}^{m} x^{(i)}$

$x := x - \mu$

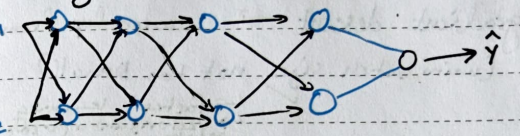
Normalize variance

$\sigma^2 = \frac{1}{m}\sum_{i=1}^{m} x^{(i)}$ **2 (element wise squaring)

$x /= \sigma$.

use same μ, σ to normaliz test set.

Vanishing gradients:



$g(z) = z$ $\quad b^{[l]} = 0$

$\hat{y} = w^{[L]} w^{[L-1]} w^{[L-2]} \cdots w^{[3]} w^{[2]} w^{[1]} x$

$w^2 > I$

$w^1 < I$

$z^1 = w^1 x$

$a^1 = g(z^1) = z^1$

$w^{[L]} = \begin{bmatrix} 0.5 & 0 \\ 1.5 & 0 \\ 0 & 1.5 \\ & 0.5 \end{bmatrix}$ $\quad \hat{y} = w^{[L]} \begin{bmatrix} 0.5 & 0 \\ 1.5 & 0 \end{bmatrix}^{L-1} x$ $\quad a^{[2]} = g(z^{[2]}) = g(w^{[2]} a^{[1]}) = g(w^{[2]} g(w^{[1]} x))$

without exploding too quicky or vanishing too quickly.

code

"He" initialization:

Initializes weights like this $w^{[L]} = \text{random value} \times \sqrt{\frac{2}{\text{size of previous layer}}}$

parameters['W' + str(l)] = np.random.randn(..) * np.sqrt(2./layer_dims[l-1])

HQ

Mini batch

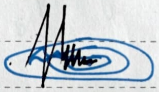
- Stochastic gradient descent → lose speedup from vectorization
- In-between (mini batch size not too small) ✓
  - ↳ fastest learning
  - ↳ Vectorization (no for loop)
  - → Make progress without processing entire training set
- Batch gradient descent. (mini batch size = m) ×
  - ↳ too long per iteration.

Mini batch size: 2^6 2^7 2^8 2^9 2^{10}

Exponentially weighted Averages

$V_t = \beta V_{t-1} + (1-\beta)\theta_t$

Gradient descent example:



↕ slower learning
↔ Faster learning

Momentum

On iteration t:

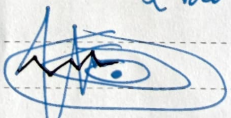
Compute dw, db on current mini batch

$V_{dw} = \beta V_{dw} + (1-\beta)\, dw$

$V_{db} = \beta V_{db} + (1-\beta)\, db$ $\quad V_0 = \beta V_{0T} + (1-\beta)\theta_t$

$w = w - \alpha V_{dw}$

RMS prop.



On t.

$S_{dw} = \beta S_{dw} + (1-\beta) dw^2$ ← small

$S_{db} = \beta S_{db} + (1-\beta) db^2$ ← large

$w = w - \alpha \frac{dw}{\sqrt{S_{dw}}}$ $\quad b = b - \alpha \frac{db}{\sqrt{S_{db}}}$

Adam Optimization Algorithm

gradient des. with momen.

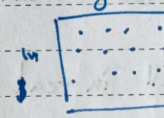
ε

Hyperparameters

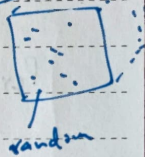
- α
- β momentum ~0.9
- $\beta_1, \beta_2, \varepsilon$ $\quad 0.9 \quad 0.999 \quad 10^{-8}$
- #layers
- # of hidden units
- learning rate decay
- mini batch size.

3

hyperparameter 2

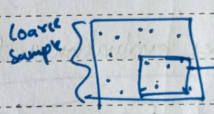


hyper 3.

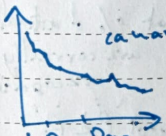


random

Right scale



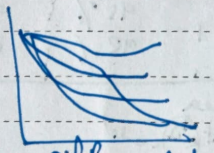
Pandas



Change the rate for single model

Pay attention to One

Caviar



Diff models

Train more models in parallel

* Batch Norm
* Softmax Activation Function: $y_t = e^{-z}$

HQ

Deep Learning.ai - Coursera

course 3. Structuring Machine learning Projects.

Orthogonalization

issues

Fit training set well on cost function → Bigger Network, Adam.

↓

Fit dev set well on cost function → Regularization, Bigger training set.

↓

Fit test set well on cost function → Bigger dev set.

↓

Performs well in real world → change dev set or cost function.

Using a single number evaluation metric.

Classifier	Precision	Recall	f1 score
A	95%	90%	92.4%
B	98%	85%	91.0%

Precision: of examples recognised as cats, what % actually are cats.

Recall: what % of actual cats are correctly recognised.

f1 score = average of P & R

$\frac{2}{\frac{1}{P}+\frac{1}{R}}$ harmonic mean.

idea → experiment → code

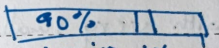
Devset + Single real number evaluation metric

↓↓

speeds up iteration.

test train dev.

For big data.



dataset examples.

× Metric + dev: Pref A

Alg A 3% error. → but gives vulgar image.

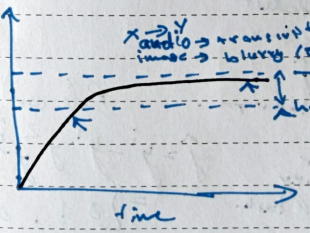
Alg B 5% error. You & users: B ✓

$$\text{Error} = \frac{1}{\sum w^{(i)}} \frac{1}{m_{dev}} \sum_{i=1}^{m_{dev}} w^{(i)} \left\{ y^{(i)}_{pred} \neq y^{(i)} \right\}$$

($y^{(i)}_{pred}$: predicted value (0/1))

$$w^{(i)} = \begin{cases} 1 & \text{if } x^{(i)} \text{ is non-vulgar} \\ 0 & \text{if } x^{(i)} \text{ is porn} \end{cases}$$

Comparing to human level performance.

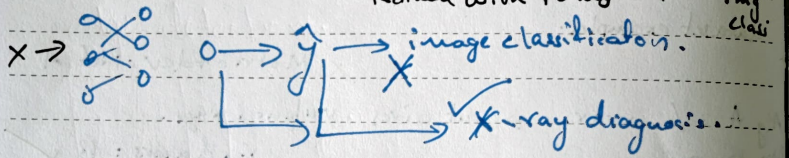


* Transfer learning

from one model to another

↳ transfer a model trained for one purpose to another with very small training data

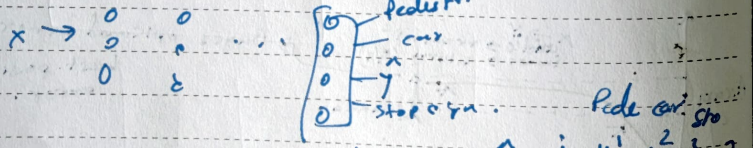
trained with images for img classi



→ trained for speech recog ✗

↳ edge word recog. ✓

Multi task transfer learning.



one neural network used for many tasks.

Pede car sto

$\hat{y} = y^1 = y^1\ y^2\ y^3 \ldots]$

$= [0\ 1\ 1\ 0 \ldots$

End-to-End deep learning.

audio → pipelines → transcript

audio → transcript

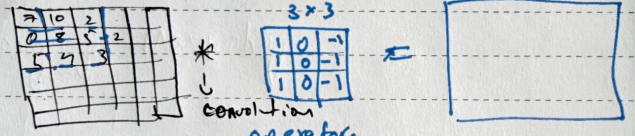
more data needed to train.

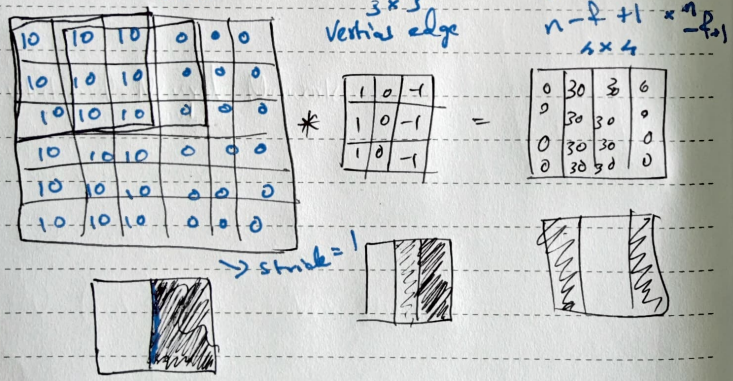
Deep Learning.ai - Coursera

Course 4 - Convolutional Neural Networks

Computer Vision

edge detection.



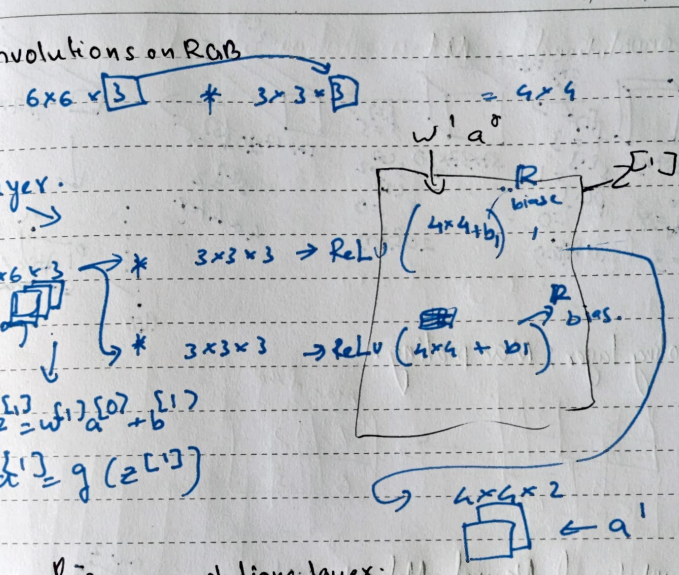


Padding

Strided convolution

stride jump = 2

Convolutions on RGB



$z^{[1]} = w^{[1]} a^{[0]} + b^{[1]}$

$a^{[1]} = g(z^{[1]})$

If layer l is a convolution layer:

$f^{[l]}$ = filter size

$p^{[l]}$ = padding

$s^{[l]}$ = stride

$n_c^{[l]}$ = number of filters.

input $n_H^{[l-1]} \times n_W^{[l-1]} \times n_c^{[l-1]}$

output $n_H^{[l]} \times n_W^{[l]} \times n_c^{[l]}$

$n_W^{[l]} = \frac{n_W^{[l-1]} + 2p^{[l]} - f^{[l]}}{s^{[l]}} + 1$

Each filter is $f^{[l]} \times f^{[l]} \times n_c^{[l-1]}$

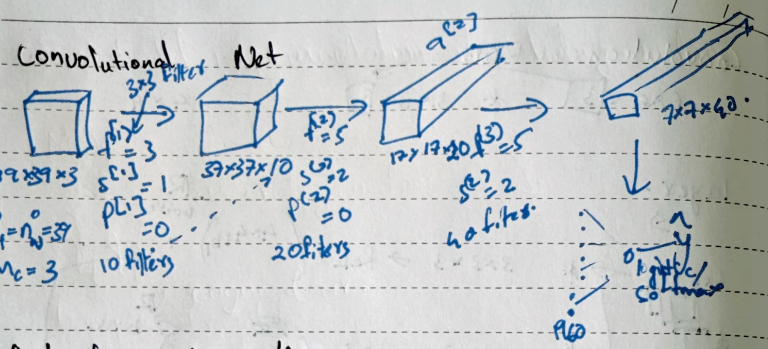
Activations: $a^{[l]} \rightarrow n_H^{[l]} \times n_W^{[l]} \times n_c^{[l]}$ $\quad A^{[l]} : m \times n_H^{[l]} \times n_W^{[l]} \times n_c^{[l]}$

weights: $f^{[l]} \times f^{[l]} \times n_c^{[l-1]} \times n_c^{[l]}$ $\quad n_c \times n_H \times n_W$.

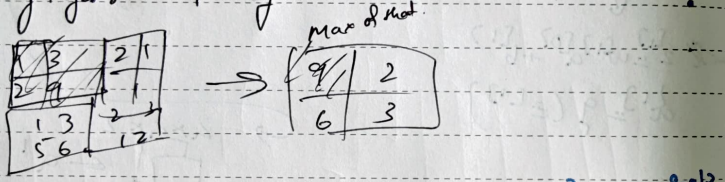
bias: $n_c^{[l]} - (1,1,1,\ldots,n_c^{[l]})$

Filters in layer l.

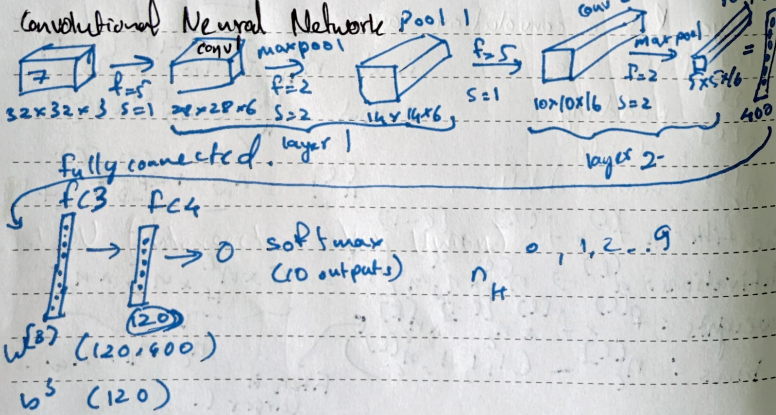
Convolutional Net



Pooling layer: Max pooling



Convolutional Neural Network



	Activation Shape	Activation Size	#parameters
input	(32, 32, 3)	3072	0
conv 1 (f=5 s=1)	(28, 28, 6)	4704	456
Pool 1	(14, 14, 6)	1176	0
conv 2 (f=5, s=1)	(10, 10, 16)	1600	2416
Pool 2	(5, 5, 16)	400	0
FC 3	(120, 1)	120	48120
FC 4	(84, 1)	84	10164
softmax	(10, 1)	10	850

Why Convolutions

Parameter sharing : eg : a feature detector

sparsity of connections.

Diff CNN

LeNet 5 → short.

Alex Net → long

VGG-16 → 16 layers.

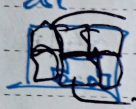
ResNets

INCEPTION nets

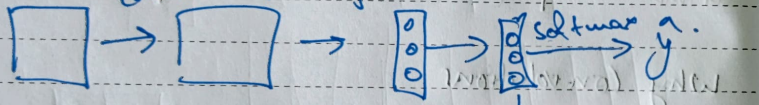
Mobile Nets : less computational resource required.

Convolutional implementation of Sliding windows

Makes use of convolution

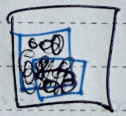


each sliding window.



$\hat{y} = \begin{bmatrix} p_c \\ b_x \\ b_y \\ b_h \\ b_w \\ c_1 \\ c_2 \\ c_3 \end{bmatrix}$ → if any object; bounding box; Ped; car; motorcycle. } class

IOU (intersection over Union)



IOU = $\frac{\text{size of } \cap}{\text{size of } \cup}$

"correct" if IOU ≥ 0.5

Non-max suppression example

discard all boxes with $p_c \leq 0.6$.

if remaining boxes → Pick box with highest p_c as prediction.

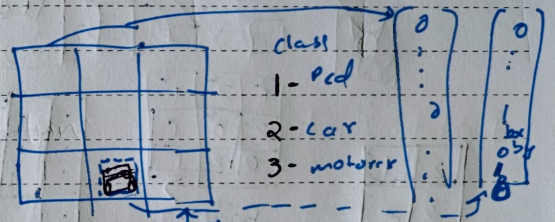
→ IoU ≥ 0.5 → Discard

Anchor Box

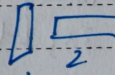
$y = \begin{Bmatrix} p_c \\ \vdots \\ p_c \end{Bmatrix}$ ← Anchorbox 1 ← Pedestrian

← car

YOLO Training



$y = \begin{bmatrix} p_c \\ b_x \\ b_y \\ b_h \\ b_w \\ c_1 \\ c_2 \\ c_3 \\ p_c \\ \vdots \end{bmatrix}$ 3×3 → 3×3×16



→ conv net → 3×3×16

100×100×3

Predicting

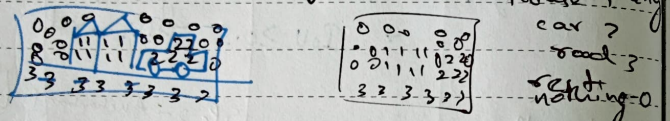
→ $y = \begin{Bmatrix} 0 \\ \vdots \\ ? \end{Bmatrix}$

HQ

Semantic Segmentation

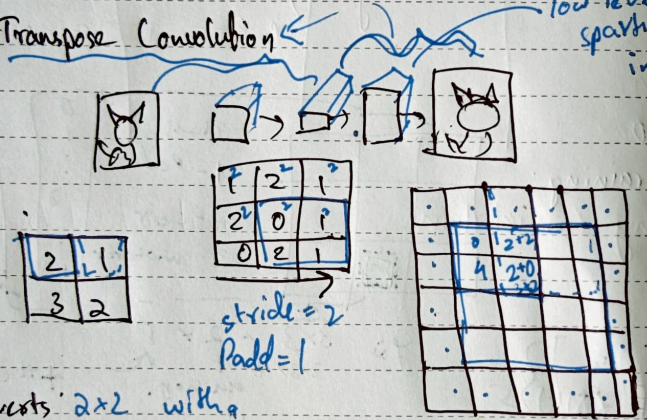
Per pixel classification

classify each pixel as part of house / sky / car / road / nothing



Transpose Convolution

low-level spatial info



converts 2×2 with a 3×3 filter to a 4×4.

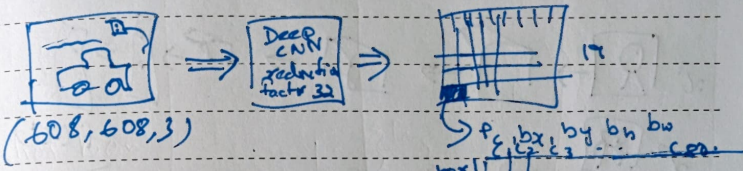
- Take left corner → multiply to all of filter, and intersect to o/p

Stride 2: with next 1

then repeat. at cross section add the values

Yolo - code based.

encoding — 5 anchor boxes: (19, 19, 5, 85)



(608, 608, 3)

scores = $p_c \times c_p$.

$= p_c \times \begin{bmatrix} c_1 \\ c_2 \\ \vdots \\ c_{80} \end{bmatrix} = \begin{bmatrix} p_c c_1 \\ p_c c_2 \\ \vdots \end{bmatrix} = \begin{bmatrix} 0.12 \\ 0.4 \\ 0.3 \\ \vdots \end{bmatrix}$ box 1

box 1: | p_c | b_x | b_y | b_h | b_w | c_1 c_2 ... c_{80} |

→ find max score 0.44

box: b_x b_y b_h b_w

class = c_3 ("car")

find class by each box. results many bounding box

→ set a threshold

→ filter

Non max Suppression

box-confidence: (19, 19, 5, 1) → p_c (if any object)

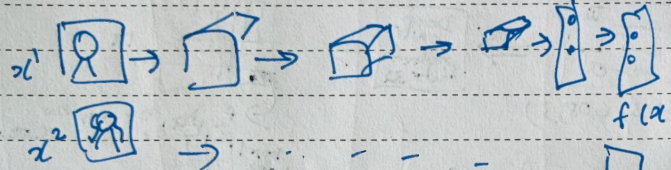
boxes: (19, 19, 5, 4) midpt & dimension (b_x, b_y, b_h, b_w)

box-class-probs: class prob for (80) for each of the boxes per cell

⇒ IoU

Face recognition

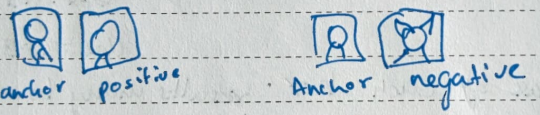
Siamese network.



$d(x_1, x_2) = \|f x_1^1 - f x_2\|^2$ is

⇑ small if same

Triplet loss function



$$\underbrace{\|f(A) - f(P)\|^2}_{d(A,P)} + \alpha \leq \underbrace{\|f(A) - f(N)\|^2}_{d(A,N)}$$

$$\underbrace{\|f(A) - f(P)\|^2}_{0} - \underbrace{\|f(A) - f(N)\|^2}_{0} + \alpha \leq 0$$

↳ Margin

loss function $f(\text{img}) = \vec{0}$

Given 3 images A, P, N

$$\mathcal{L}(A,P,N) = \max(\|f(A) - f(P)\|^2 - \|f(A) - f(N)\|^2 + \alpha, 0)$$

$$J = \sum_{i=1}^{m} \mathcal{L}(A^{(i)}, P^{(i)}, N^{(i)})$$

Training: 10k pictures of 1k pictures. A, P

Neural Style Transfer

3 ⇑ cost function = 1 + 2.

1 content cost function

2 Style cost function

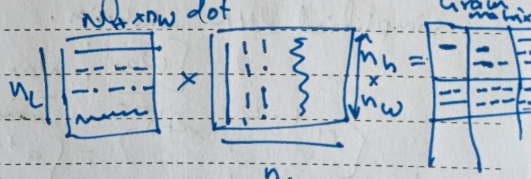
$$J(G) = \alpha J_{content}(C, G) + \beta J_{style}(S, G)$$

shallow layers of conv Net → detect edges & simple textures

deep " → objects & complex textures

Find the middle layer for better.

Gram Matrix



Deep Learning.ai - coursera

Sequence Models - course 5

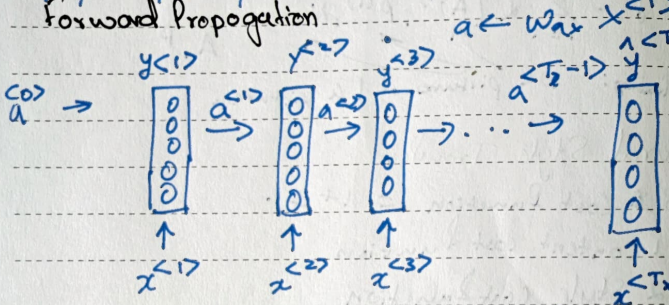
"Harry and Hermione cast new spell"

x^1 x^2 x^3 ... x^4 ... x^t

y^1 y^2 y^3 0 0 ... 1

forward Propogation

$a \leftarrow W_{ax} x^{<1>}$



$a^{<0>} = 0$

$a^{<1>} = g_1(W_{aa} a^{<0>} + W_{ax} x^{<1>} + b_a)$ ← tanh / ReLU

$\hat{y}^{<1>} = g_2(W_{ya} a^{<1>} + b_y)$ ← Sigmoid

$a^{<t>} = g(W_{aa} a^{<t-1>} + W_{ax} x^{<t>} + b_a)$

$\hat{y}^{<t>} = g(W_{ya} a^{<t>} + b_y)$

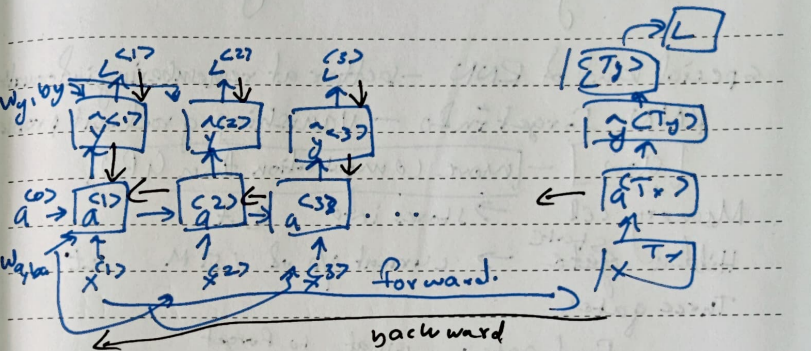
simplified RNN notation.

$[W_{aa} \vdots W_{ax}] = W_a$ (100, 10000)

$\begin{bmatrix} a^{<t-1>} \\ x^{<t>} \end{bmatrix} = \begin{bmatrix} a^{<t-1>} \\ x^{<t>} \end{bmatrix}$ (100, 1, 10000)

$W_{aa} a^{<t-1>} + W_{ax} x^{<t>} = W_a \begin{bmatrix} a^{<t-1>} \\ x^{<t>} \end{bmatrix}$

backward.



$L^{<T>}(\hat{y}^{<t>}, y^{<t>}) = -y^{<t>} \log \hat{y}^{<t>} - (1-y^{<t>}) \log(1-\hat{y}^{<t>})$

$L(\hat{y}, y) = \sum_{t=1}^{T_y} L^{<t>}(\hat{y}^{<t>}, y^{<t>})$ ←

Language model.

→ assign probability to a sentence → how likely a sentence is to appear in real world.

"cats average" word1 word2 ... model learns 1 comes next.

$P(\text{word1}) * P(\text{word2} \mid \text{word1}) * P(\text{word3} \mid \text{word1, word2})$

give training corpus

then prediction → good.

GRU

HQ

LSTM - Long Short term Memory

Special kind of RNN - better at remembering information

RNN - forget info - vanishing gradient problem

Slow - more computation than GRU

Memory cell → stores info c_t

Hidden ~~gate~~ state → current o/p of LSTM a_t

Three gates

- Forget gate: what to forget
- Update gate: what to add to mem
- Output gate: what to o/p

Steps

1. LSTM looks at
   - previous hidden state $a_{(t-1)}$
   - previous memory cell $c_{(t-1)}$
   - current input x_t
2. It computes
   - forget gate → How much of $c_{(t-1)}$ to keep
   - input gate + candidate → how much new info to add
   - output gate → what part of memory to o/p

Updates

- new memory cell c_t is computed
- new hidden state a_t is computed

Bidirectional RNN